



<https://haojian.github.io/DSC204A26SP/>

DSC 204A: Scalable Data Systems

Fall 2026

Instructor: Haojian Jin

Where We Are

Motivations, Economics, Ecosystems,
Trends



Networking

Storage

Part3: Compute

Datacenter
networking

Collective
communication

(Distributed) File
Systems / Database

Cloud storage

**Distributed
Computing**

**Big data
processing**

Let's Focus On: Multi-node Distributed Systems

We have two primary problems to solve in real systems

1. How to Distribute Data (we'll not cover, also not in exam)

- Read DDIA
- Read GFS paper
- Read BigTable paper

2. How to Distribute Compute (we'll cover in lectures)

- Batching Processing
- Streaming Processing

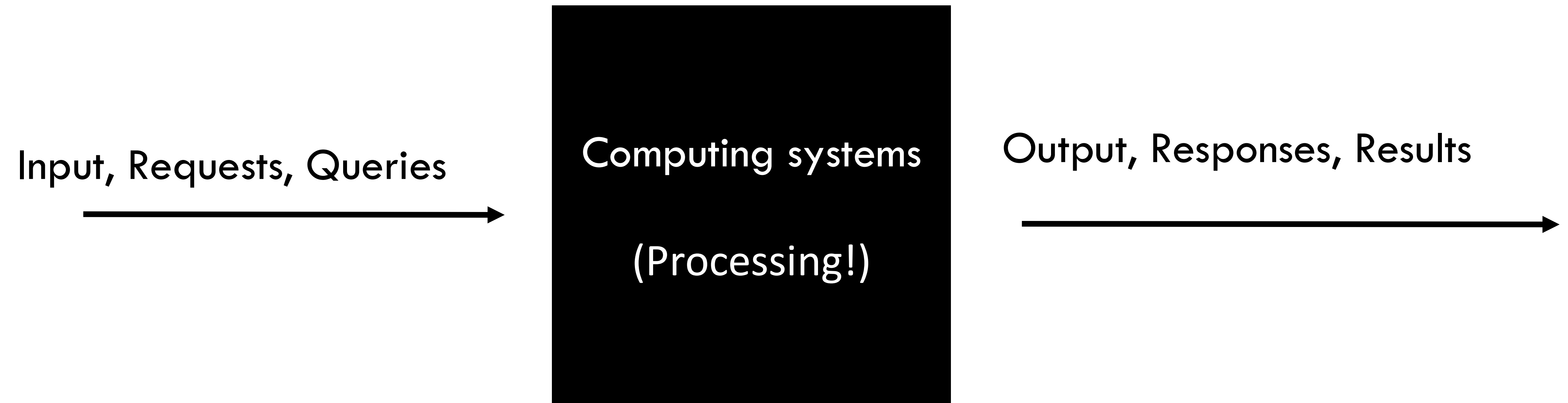
How to Analyze Distributed Systems

- Scalability
 - Data volume
 - Read/Write/Compute load
- Consistency and correctness
 - Read / Write sees consistency data
 - Compute produce correct results
- Fault tolerance / high availability
 - When one fails, another can take over.
- Latency and throughput
 - Distribute machines worldwide.
 - Reduce network latency.

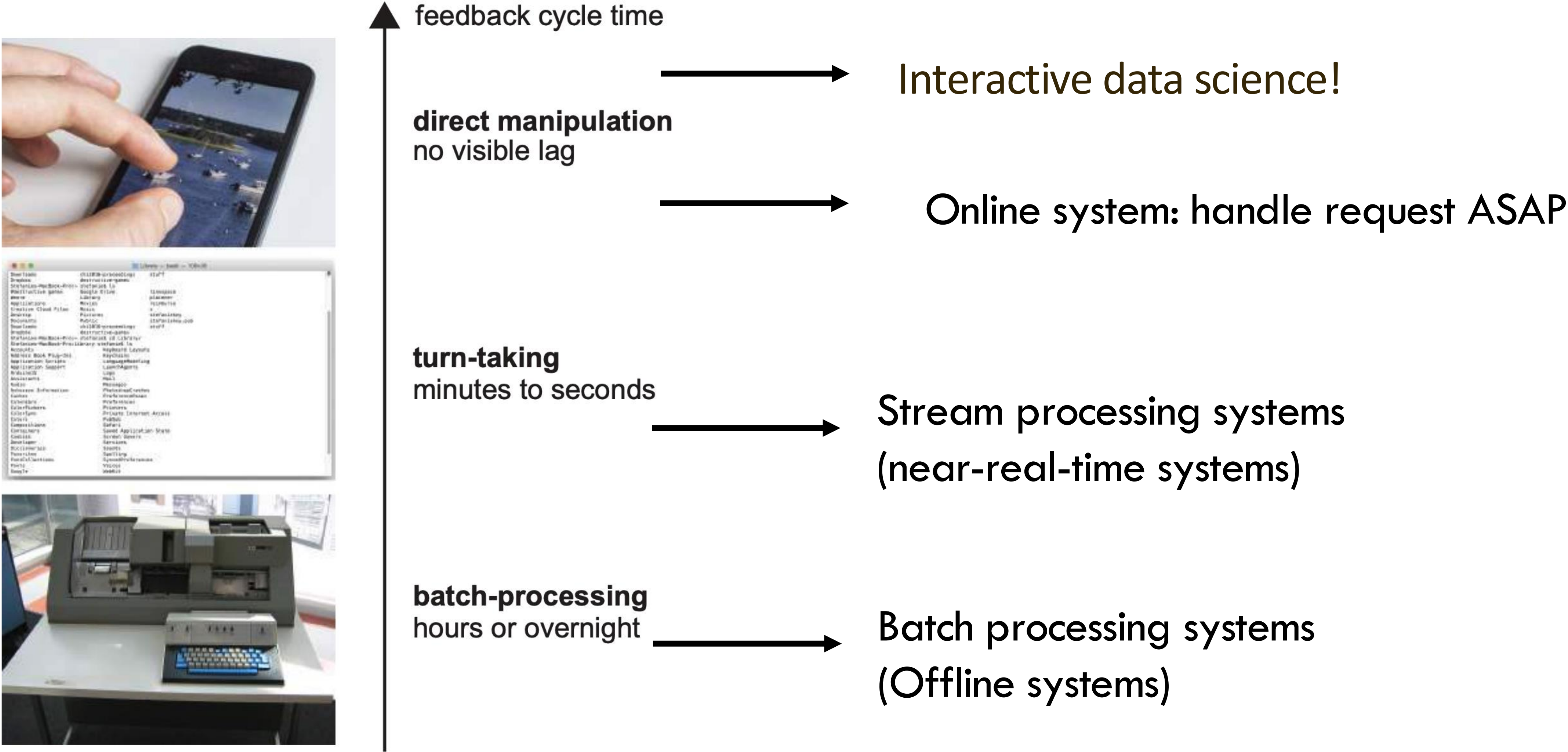
Today's topic: Batch Processing

- Overview
- IO & Unix pipes
- MapReduce
- Beyond MapReduce

Basic Computing System Paradigm



Processing latency



Today's topic: Batch Processing

- Overview
- IO & Unix pipes: the first batching processing system
- MapReduce
- Beyond MapReduce

Shell example

“*****rc”

Run commands



```
hao@HaoPC: /mnt/e/projects x + v
hao@HaoPC:/mnt/e/projects/projects/courses/dsc204a-w24$ ls -lah ~/.
total 256K
drwxr-xr-x 30 hao hao 4.0K Feb 16 14:01 .
drwxr-xr-x 3 root root 4.0K Jul 2 2021 ..
drwxr-xr-x 2 hao hao 4.0K Sep 1 00:10 .aws
drwxr-xr-x 5 hao hao 4.0K Nov 4 2021 .azure
-rw-r--r-- 1 hao hao 25K Feb 16 14:01 .bash_history
-rw-r--r-- 1 hao hao 220 Jul 2 2021 .bash_logout
-rw-r--r-- 1 hao hao 4.4K Jan 1 22:17 .bashrc
-rw-r--r-- 1 hao hao 21K Apr 13 2023 .boto
drwxr-xr-x 3 hao hao 4.0K Apr 29 2023 .bundle
drwxr-xr-x 11 hao hao 4.0K Jun 18 2023 .cache
drwxr-xr-x 12 hao hao 4.0K Aug 16 2023 .config
drwxr-xr-x 3 hao hao 4.0K Nov 21 2022 .cupy
drwxr-xr-x 3 hao hao 4.0K Oct 3 2021 .eclipse
drwxr-xr-x 4 hao hao 4.0K Apr 29 2023 .gem
-rw-r--r-- 1 hao hao 96 Jun 13 2023 .gitconfig
drwxr-xr-x 2 hao hao 4.0K Jun 22 2022 .gnupg
drwxr-xr-x 3 hao hao 4.0K May 12 2023 .gsutil
drwxr-xr-x 3 hao hao 4.0K Nov 22 2022 .ipython
drwxr-xr-x 2 hao hao 4.0K Nov 22 2022 .jupyter
drwxr-xr-x 3 hao hao 4.0K Jan 1 2023 .kube
drwxr-xr-x 2 hao hao 4.0K Jul 2 2021 .landscape
drwxr-xr-x 7 hao hao 4.0K Jan 6 02:09 .local
-rw-r--r-- 1 hao hao 0 Feb 23 09:36 .motd_shown
drwxr-xr-x 2 hao hao 4.0K Apr 28 2023 .ngrok
-rw-r--r-- 1 hao hao 18 Jun 22 2023 .node_repl_history
drwxr-xr-x 7 hao hao 4.0K Apr 30 2023 .npm
drwxr-xr-x 3 hao hao 4.0K Dec 30 2022 .nv
drwxr-xr-x 8 hao hao 4.0K Apr 28 2023 .nvm
-rw-r--r-- 1 hao hao 807 Jul 4 2023 .profile
drwxr-xr-x 23 hao hao 4.0K Aug 18 2023 .pycharm_helpers
-rw-r--r-- 1 hao hao 4.1K Aug 25 22:12 .python_history
drwxr-xr-x 2 hao hao 4.0K Nov 28 2022 .ray
drwxr-xr-x 4 hao hao 4.0K Jan 1 22:21 .rbenv
drwxr-xr-x 2 hao hao 4.0K Feb 20 2023 .skyplane
drwxr-xr-x 2 hao hao 4.0K Dec 10 12:23 .ssh
-rw-r--r-- 1 hao hao 0 Jul 24 2021 .sudo_as_admin_successful
drwxr-xr-x 2 hao hao 4.0K Dec 10 2022 .vim
-rw-r--r-- 1 hao hao 32K Jan 7 18:34 .viminfo
-rw-r--r-- 1 hao hao 355 Nov 4 2021 .vimrc
drwxr-xr-x 5 hao hao 4.0K Mar 12 2022 .vscode-server-insiders
-rw-r--r-- 1 hao hao 215 May 15 2023 .wget-hsts
-rw-r--r-- 1 hao hao 0 Sep 12 23:26 calculate_flops.py
drwxr-xr-x 1 hao hao 3.6K Sep 13 00:39 estimate_throughput.py
drwxr-xr-x 6 hao hao 4.0K Jun 18 2023 logs-env
drwxr-xr-x 12 hao hao 4.0K Aug 21 2023 my_site
-rw-r--r-- 1 hao hao 646 Sep 2 01:48 perf_model.py
-rw-r--r-- 1 hao hao 333 Sep 2 02:13 test.py
```

Useful shell commands

- Shell already has a collection of rich commands
 - Some Useful commands
 - uptime, cut, date, cat, finger, hexdump, man, md5sum, quota,
 - mkdir, rmdir, rm, mv, du, df, find, cp, chmod, cd
 - uname, zip, unzip, gzip, tar
 - tr, sed, sort, uniq, ascii
 - Type “man command” to read about shell commands

What do these shell commands do?

- `cat dups.txt | sort | uniq`
- `cat dups.txt | sort -V | uniq`
- `cat dups.txt | sort -V | uniq > outfile.txt`
- `tr "a" "e" < z.txt`
- `cat z.txt | tr a e`

Batch processing with Unix Tools

```
cat /var/log/nginx/access.log | ❶  
  awk '{print $7}' | ❷  
  sort | ❸  
  uniq -c | ❹  
  sort -r -n | ❺  
  head -n 5 ❻
```

```
4189 /favicon.ico  
3631 /2013/05/24/improving-security-of-ssh-private-keys.html  
2124 /2012/12/05/schema-evolution-in-avro-protocol-buffers-thrift.html  
1369 /  
915 /css/typography.css
```

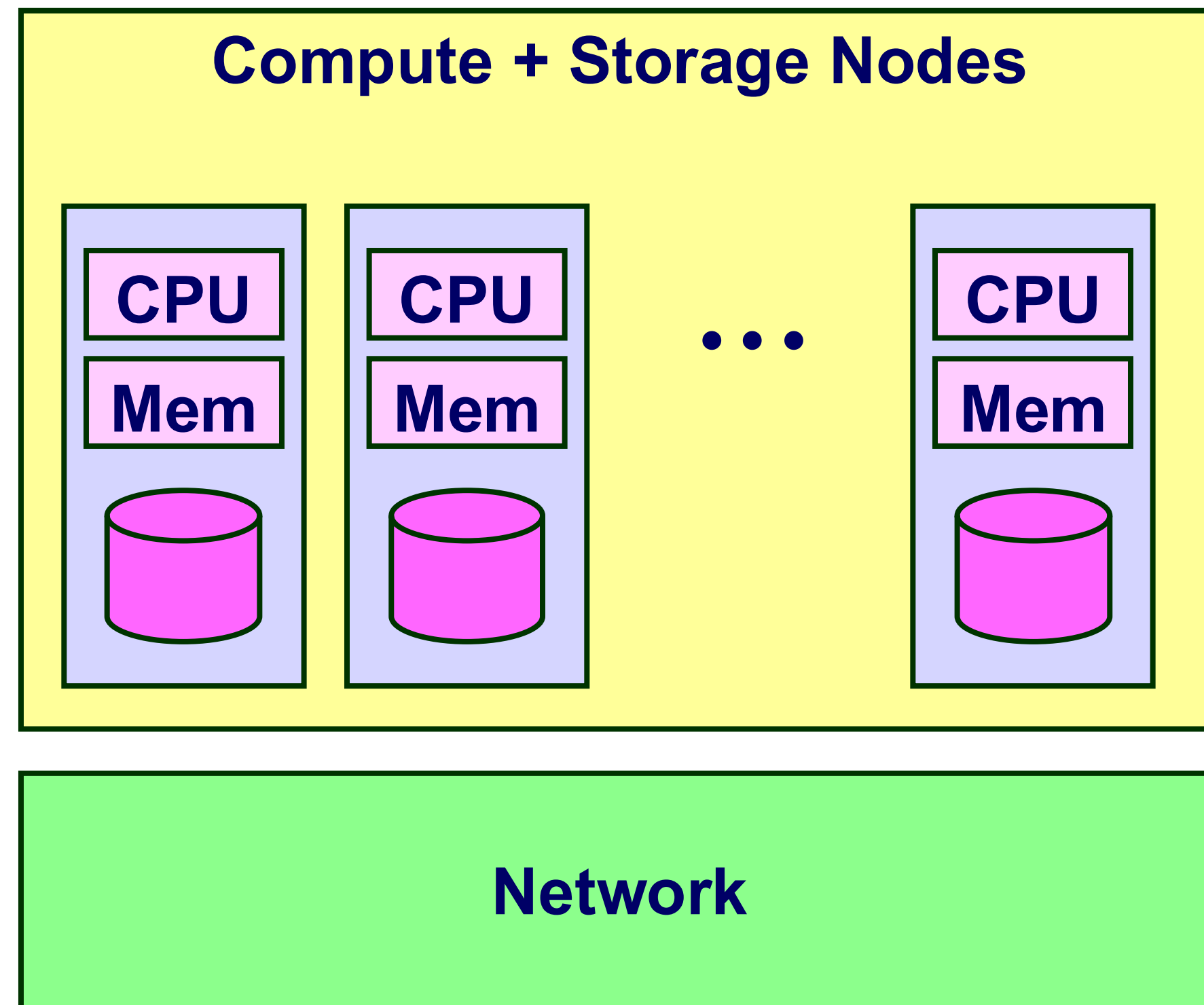
- Read the log file.
- Split each line into fields by white space, output only the 7th element (requested URL).
- Alphabetically sort
- Filter out repeated lines.
- Sort it again based on the line number (-n)
- Output the first five lines.

The biggest limitation of Unix tools is that they **run only on a single machine** — and that's where tools like Hadoop come in.

Today's topic: Batch Processing

- Overview
- IO & Unix pipes
- MapReduce
 - HDFS - infrastructure
 - Programming models (api)
 - Job execution
 - Workflow
- Beyond MapReduce

History of MapReduce/Hadoop



Compute + Storage Nodes

- Medium-performance processors
- Modest memory
- 1-2 disks

Network

- Conventional Ethernet switches
 - 10 Gb/s within rack
 - 100 Gb/s across racks

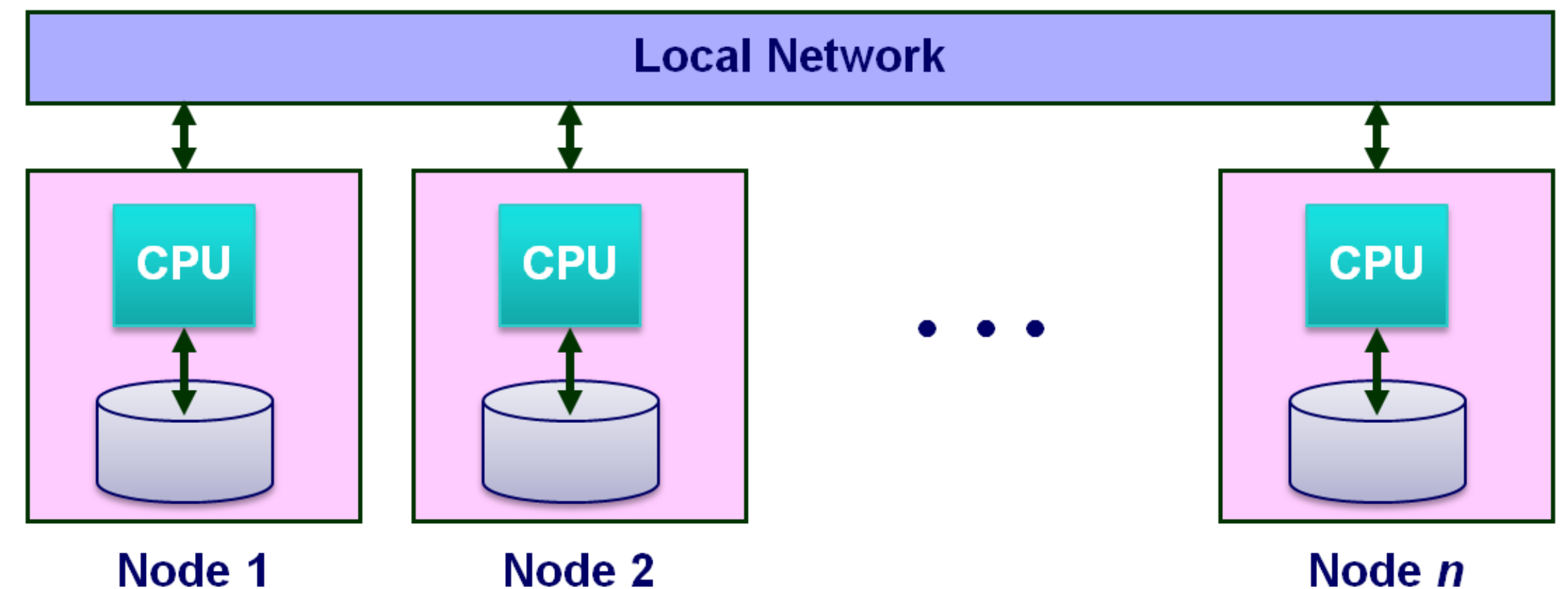
Data-Intensive System Challenge

For Computation That Accesses 1 TB in 5 minutes

- Data distributed over 100+ disks
 - Assuming uniform data partitioning
- Compute using 100+ processors
- Connected by gigabit Ethernet (or equivalent)

System Requirements

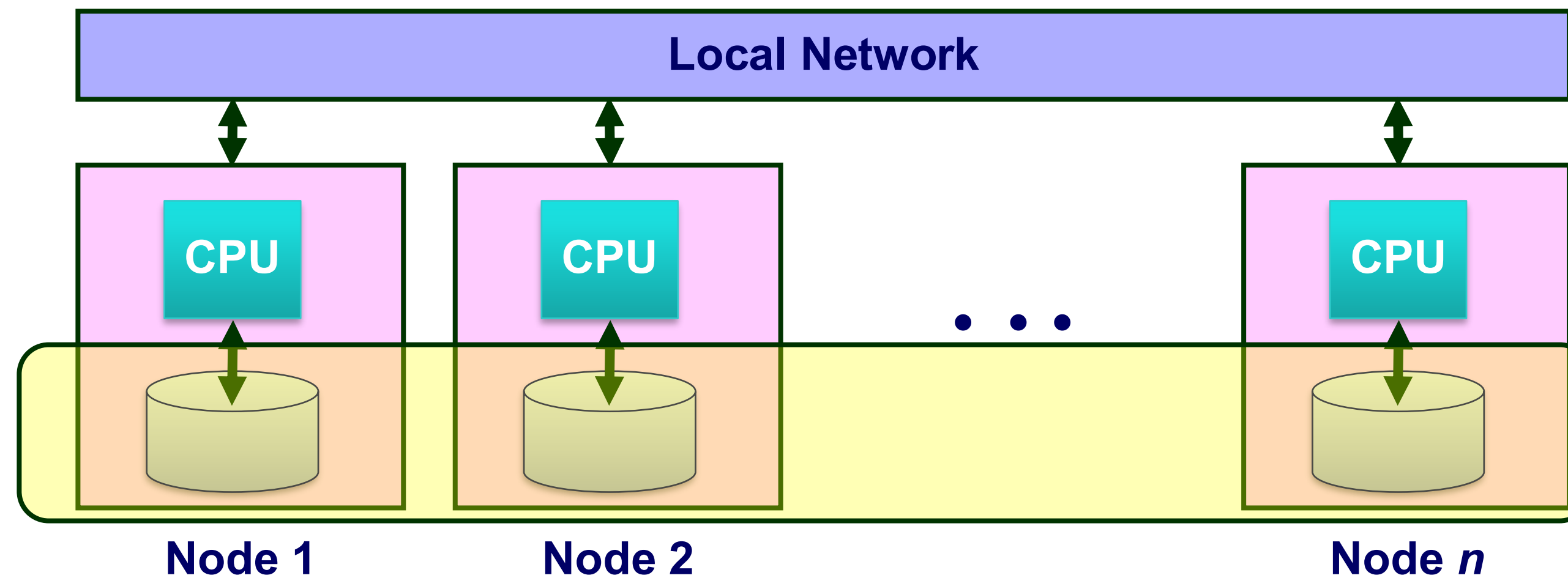
- Lots of disks
- Lots of processors
- Located in close proximity
 - Within reach of fast, local-area network





Hadoop Project

File system with files distributed across nodes



- Store multiple (typically 3 copies of each file)
 - If one node fails, data still available
- Logically, any node has access to any file
 - May need to fetch across network (ideally, leverage locality for perf.)

Map / Reduce programming environment

- Software manages execution of tasks on nodes

Today's topic: Batch Processing

- Overview
- IO & Unix pipes
- MapReduce
 - HDFS - infrastructure
 - Programming models (API)
 - Job execution (runtime)
 - Workflow
- Beyond MapReduce

MAPREDUCE: SIMPLIFIED DATA PROCESSING ON LARGE CLUSTERS

by Jeffrey Dean and Sanjay Ghemawat

Abstract

MapReduce is a programming model and an associated implementation for processing and generating large datasets that is amenable to a broad variety of real-world tasks. Users specify the computation in terms of a *map* and a *reduce* function, and the underlying runtime system automatically parallelizes the computation across large-scale clusters of machines, handles machine failures, and schedules inter-machine communication to make efficient use of the network and disks. Programmers find the system easy to use: more than ten thousand distinct MapReduce programs have been implemented internally at Google over the past four years, and an average of one hundred thousand MapReduce jobs are executed on Google's clusters every day, processing a total of more than twenty petabytes of data per day.

ANNALS OF TECHNOLOGY

THE FRIENDSHIP THAT MADE GOOGLE HUGE

Coding together at the same computer, Jeff Dean and Sanjay Ghemawat changed the course of the company—and the Internet.

By James Somers

December 3, 2018



<https://www.newyorker.com/magazine/2018/12/10/the-friendship-that-made-google-huge>

TF-IDF

TF-IDF is a measure of originality of a word by comparing the number of times a word appears in a doc with the number of docs the word appears in.

$$\text{TF-IDF} = \text{TF}(t, d) \times \text{IDF}(t)$$

Term frequency

Number of times term t appears in a doc, d

Inverse document frequency

$$\log \frac{1 + n}{1 + \text{df}(d, t)}$$

Document frequency of the term t

of documents

Count the number of occurrences of word in a large collection of documents

```
map(String key, String value):  
    // key: document name  
    // value: document contents  
    for each word w in value:  
        EmitIntermediate(w, "1");
```

```
reduce(String key, Iterator values):  
    // key: a word  
    // values: a list of counts  
    int result = 0;  
    for each v in values:  
        result += ParseInt(v);  
    Emit(AsString(result));
```

- Functional programming
- Functions are stateless
- They takes an input, processes and output a result.
- Pros and Cons?

Data models

```
map(String key, String value):  
    // key: document name  
    // value: document contents  
    for each word w in value:  
        EmitIntermediate(w, "1");
```

```
reduce(String key, Iterator values):  
    // key: a word  
    // values: a list of counts  
    int result = 0;  
    for each v in values:  
        result += ParseInt(v);  
    Emit(AsString(result));
```



map
reduce

(k1,v1)

(k2,list(v2))

→ list(k2,v2)

→ list(v2)

MapReduce Example

- Create a word index of set of documents

**Come,
Dick**

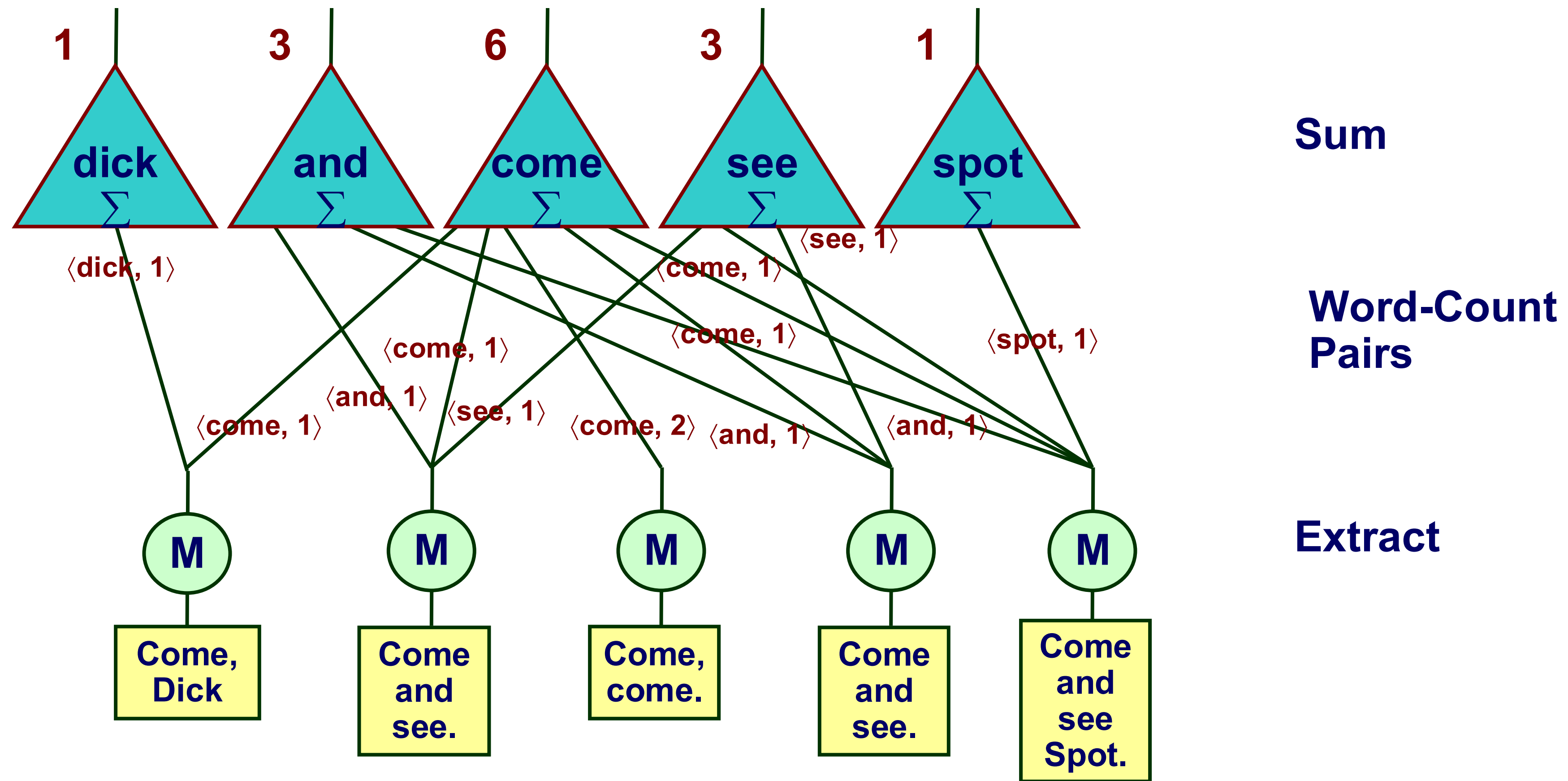
**Come
and
see.**

**Come,
come.**

**Come
and
see.**

**Come
and
see
Spot.**





- Map: generate $\langle \text{word}, \text{count} \rangle$ pairs for all words in document
- Reduce: sum word counts across documents

Discussion:

Other possible way to implement this using map-reduce?

Today's topic: Batch Processing

- Overview
- IO & Unix pipes
- MapReduce
 - HDFS - infrastructure
 - Programming models
 - Job execution
 - Workflow
- Beyond MapReduce

MapReduce Execution (Runtime)

R Output Files



R Reducers



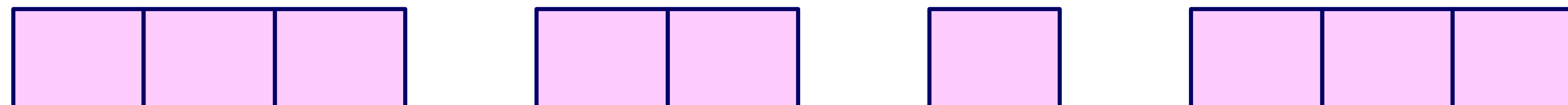
Shuffle



M Mappers

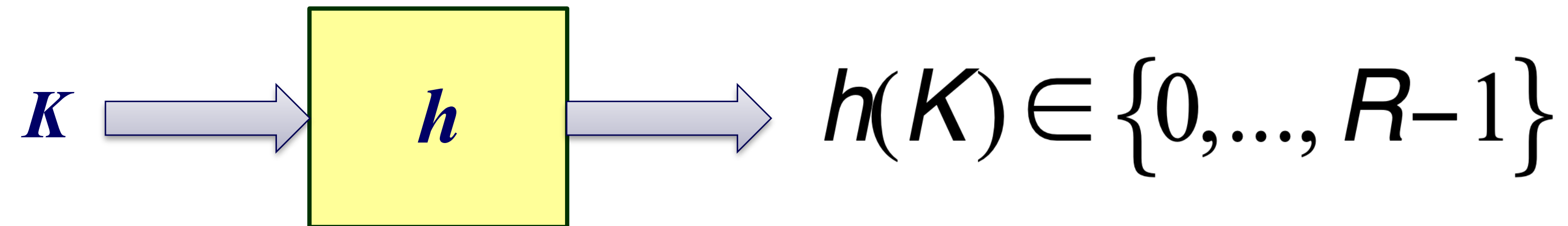


Input Files (Partitioned into Blocks)



Why do we need
shuffle?

Single Mapper

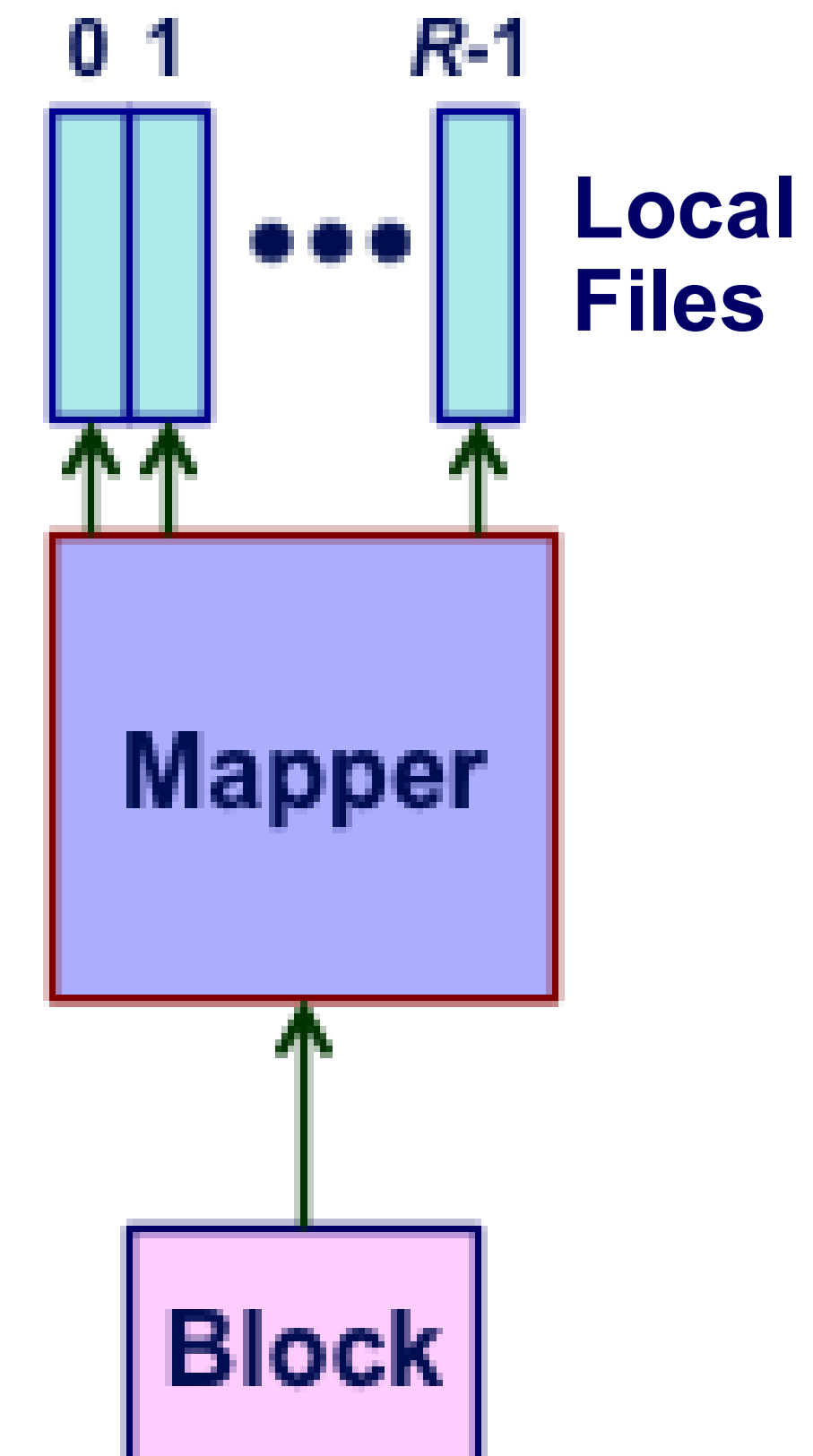


Hash Function h

- Maps each key K to integer i such that $0 \leq i < R$

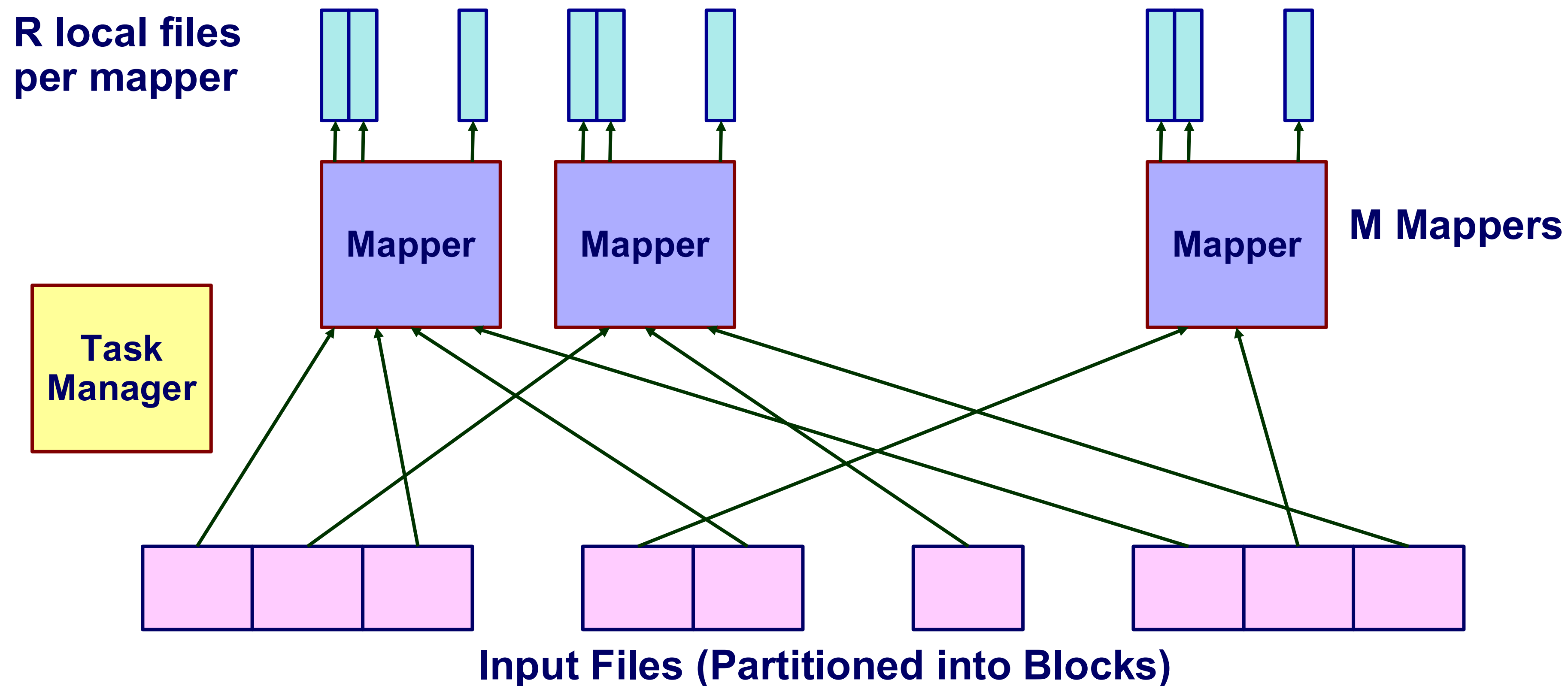
Mapper Operation

- Reads input file blocks
- Generates pairs $\langle K, V \rangle$
- Writes to local file $h(K)$



Distributed Mapper

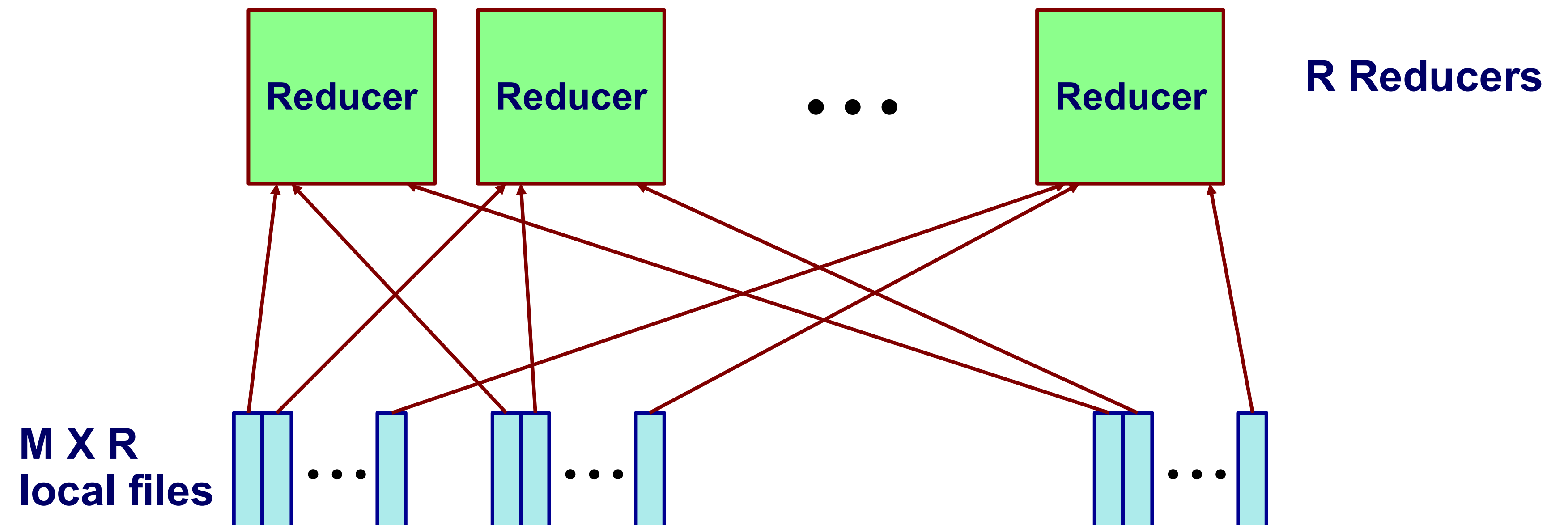
- Dynamically map input file blocks onto mappers
- Each generates key/value pairs from its blocks
- Each writes R files on local file system



Shuffling

Each Reducer:

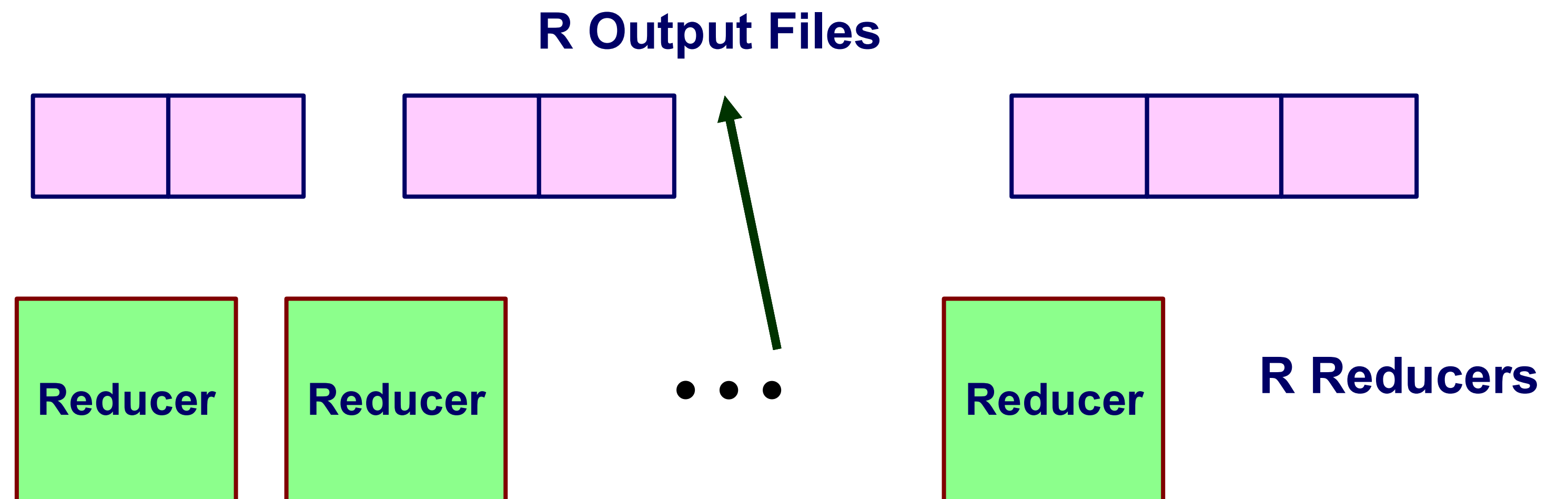
- Handles $1/R$ of the possible key values
- Most cases: just a hash function
- Need to handle fault tolerance



Reducer

Each Reducer:

- Executes reducer function for each key
- Writes output values to parallel file system

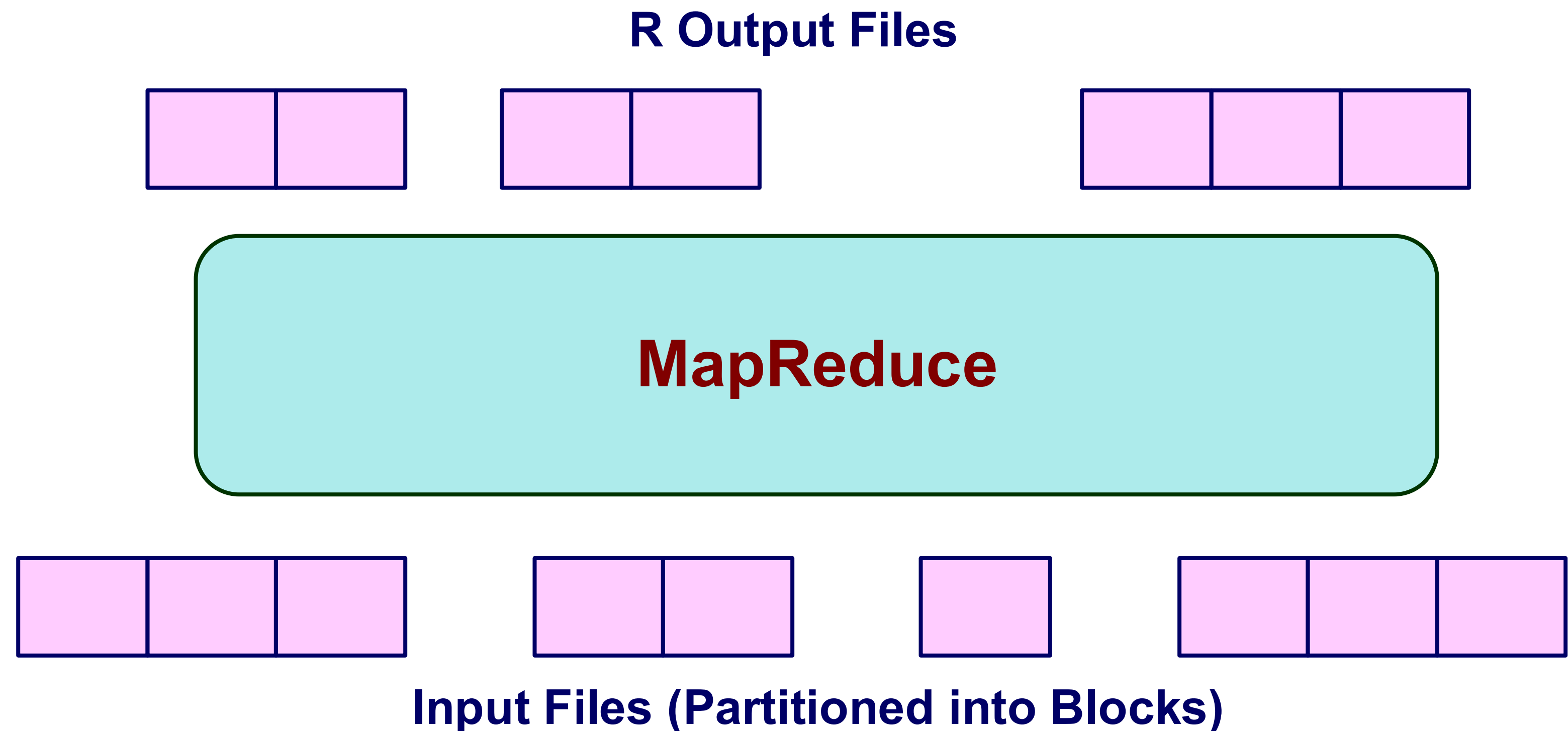


MapReduce Effect

MapReduce Step

- Reads set of files from file system
- Generates new set of files

Can iterate to do more complex processing



Today's topic: Batch Processing

- Overview
- IO & Unix pipes
- MapReduce
 - HDFS - infrastructure
 - Programming models (API)
 - Job execution (runtime)
 - MapReduce dataflow
- Beyond MapReduce

Example: Sparse Matrices with Map/Reduce

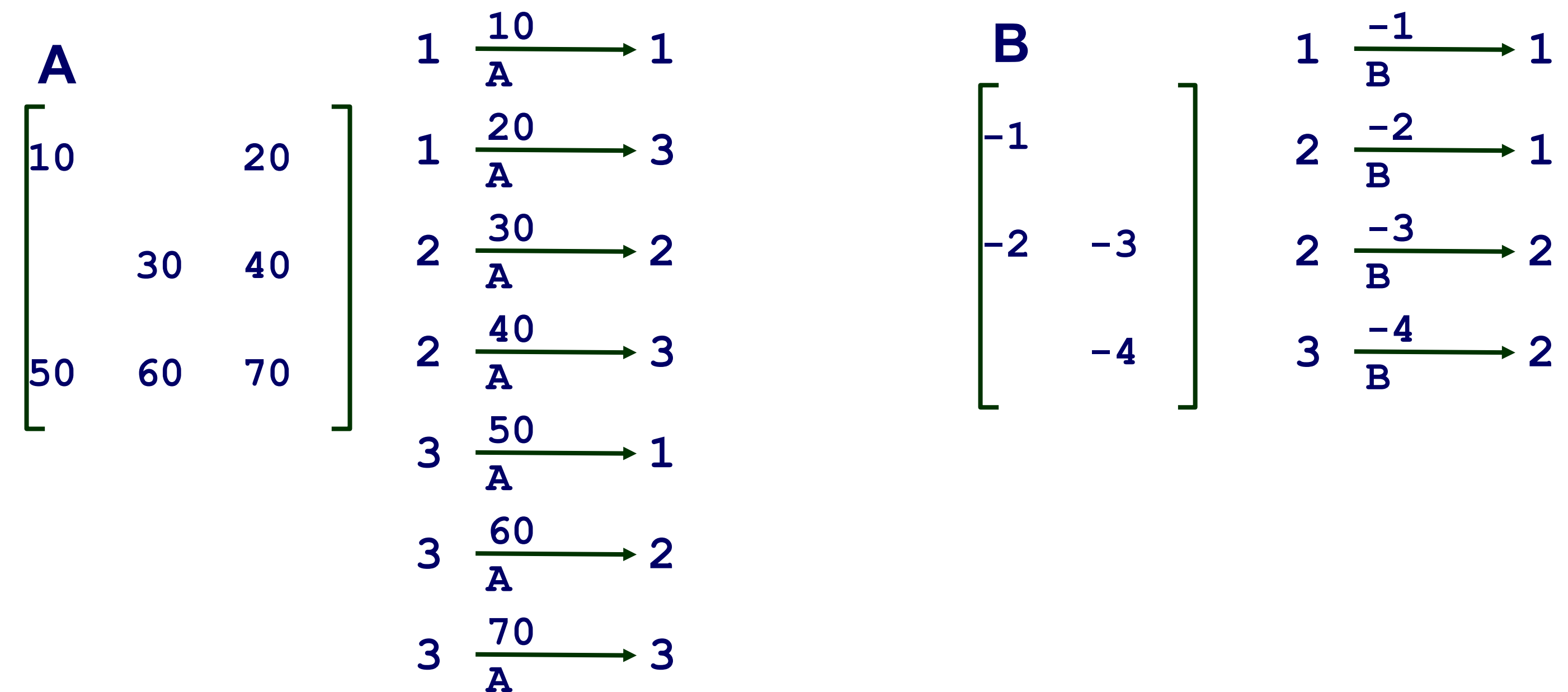
$$\begin{array}{c} \mathbf{A} \\ \begin{bmatrix} 10 & & 20 \\ & 30 & 40 \\ 50 & 60 & 70 \end{bmatrix} \end{array} \times \begin{array}{c} \mathbf{B} \\ \begin{bmatrix} -1 & \\ -2 & -3 \\ & -4 \end{bmatrix} \end{array} = \begin{array}{c} \mathbf{C} \\ \begin{bmatrix} -10 & -80 \\ -60 & -250 \\ -170 & -460 \end{bmatrix} \end{array}$$

- Task: Compute product $C = A \cdot B$
- Assume most matrix entries are 0

Motivation

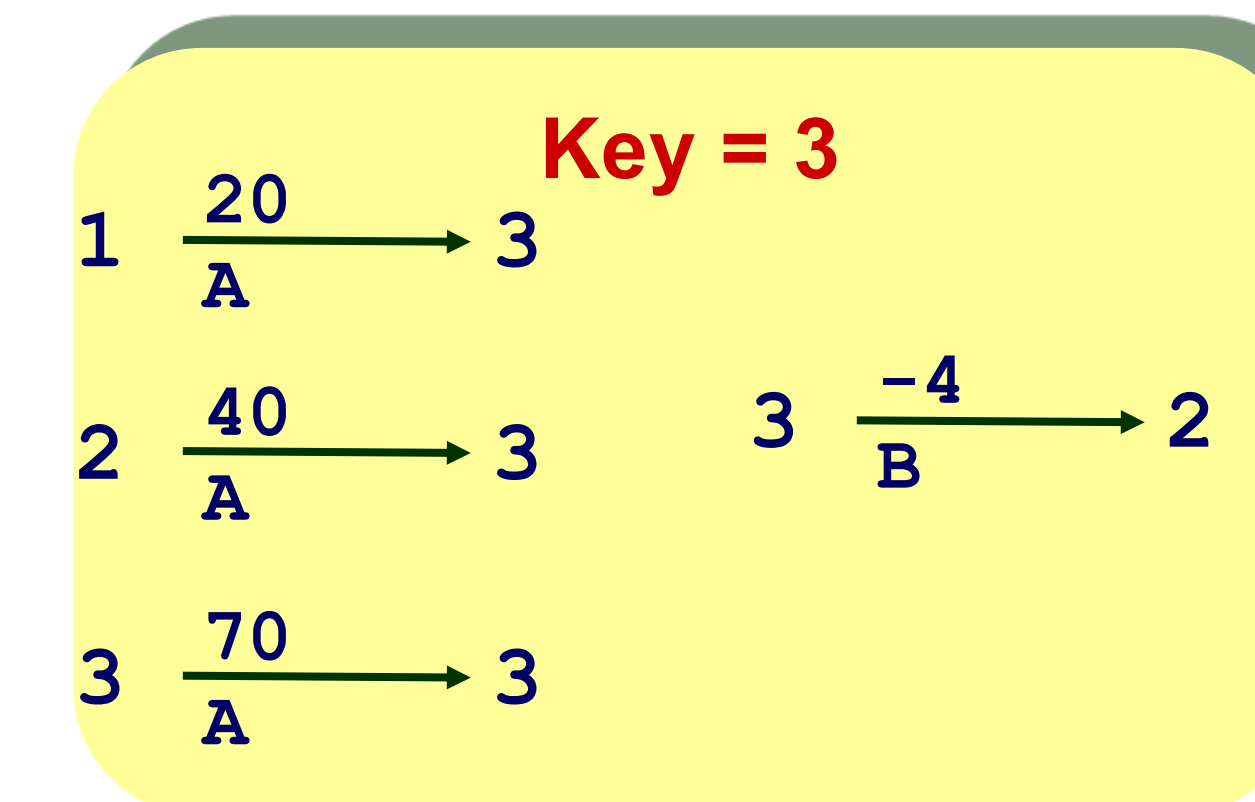
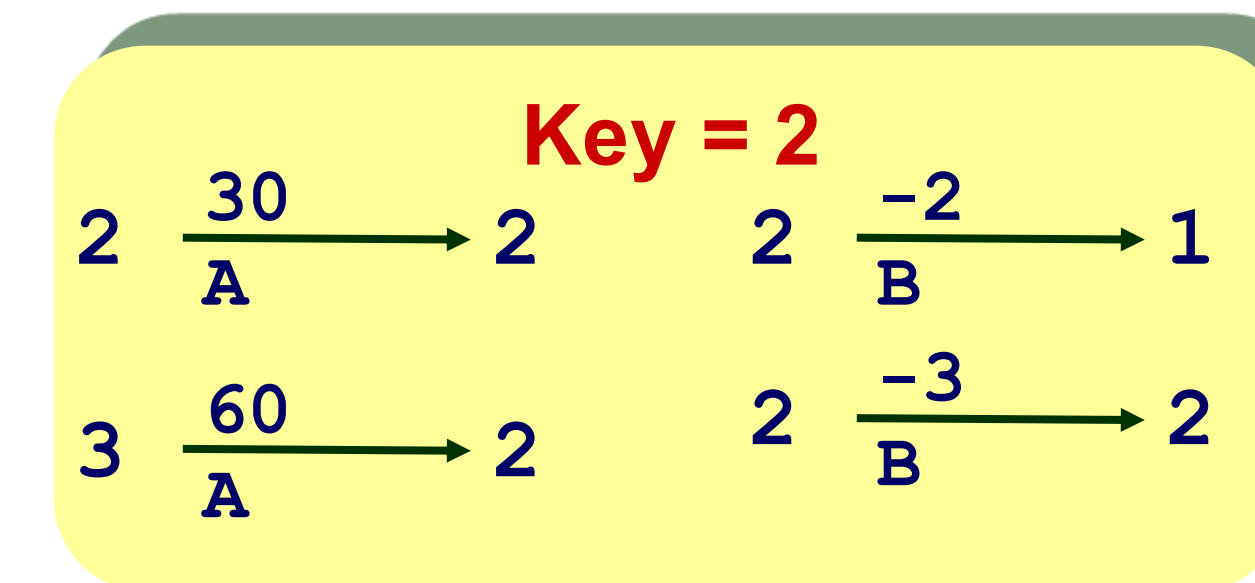
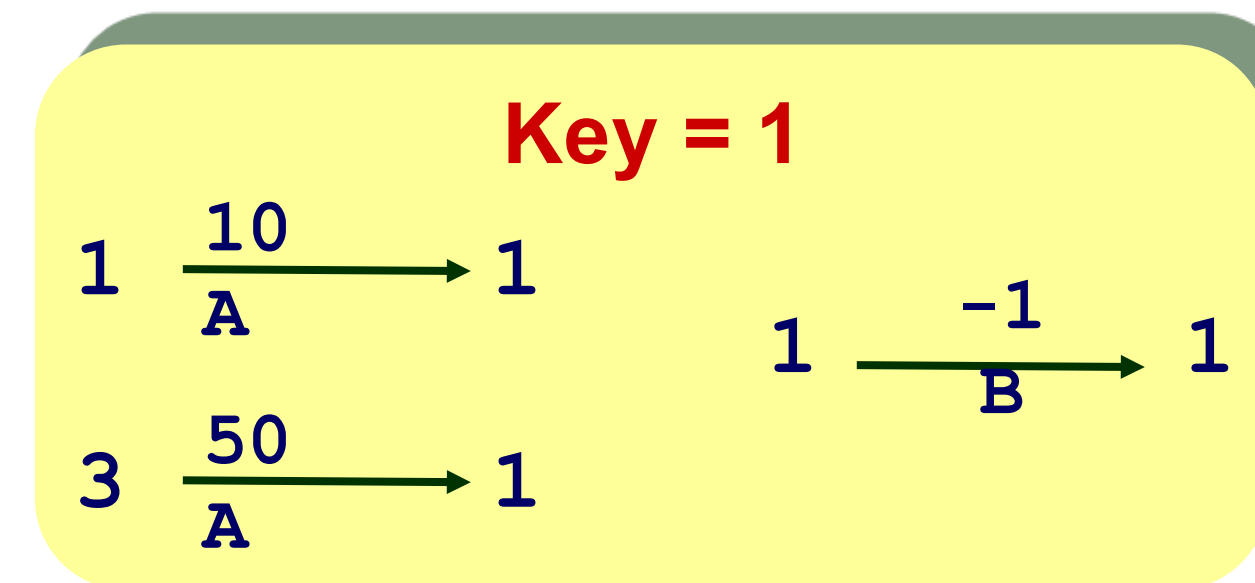
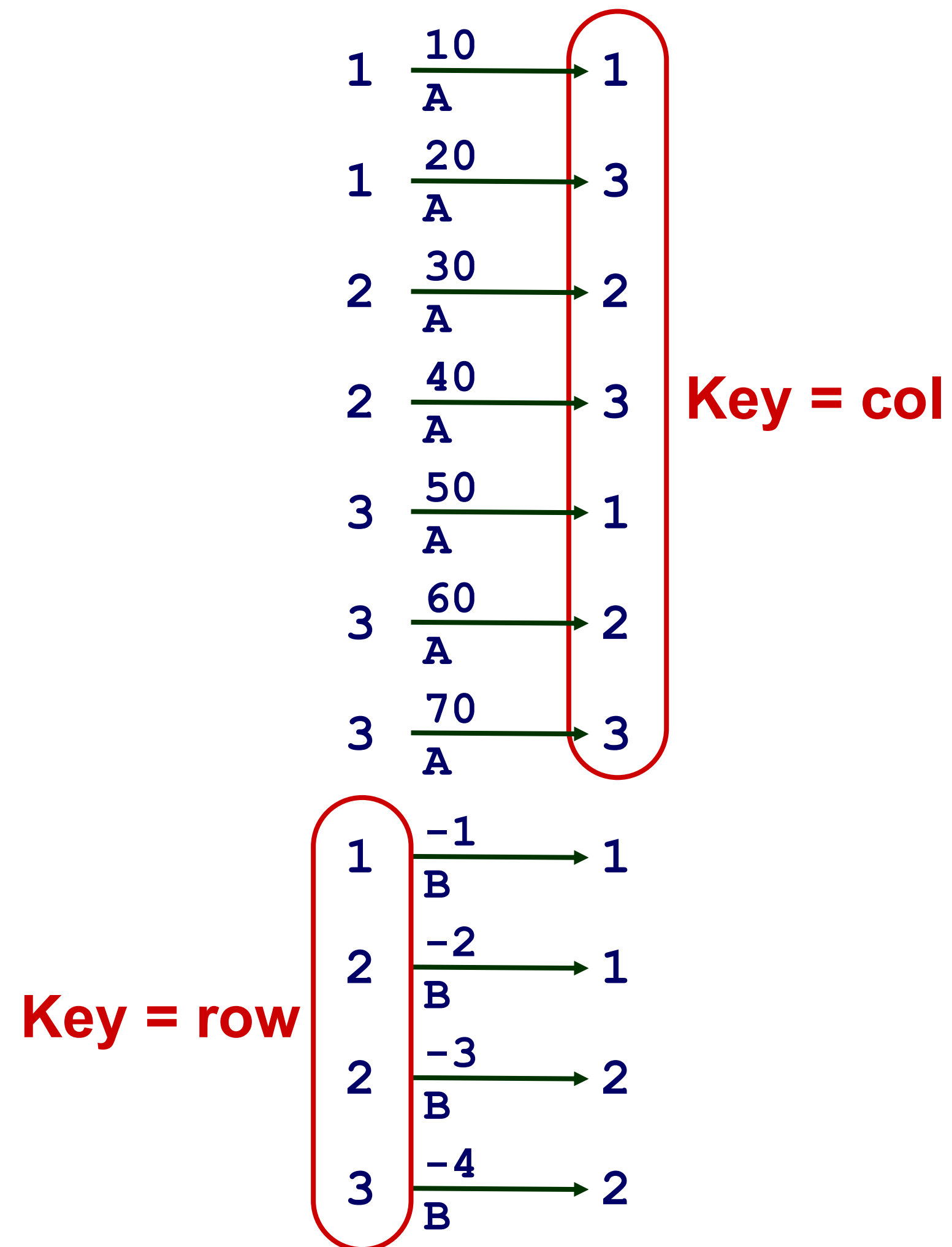
- Core problem in scientific computing
- Challenging for parallel execution
- Demonstrate expressiveness of Map/Reduce

Computing Sparse Matrix Product



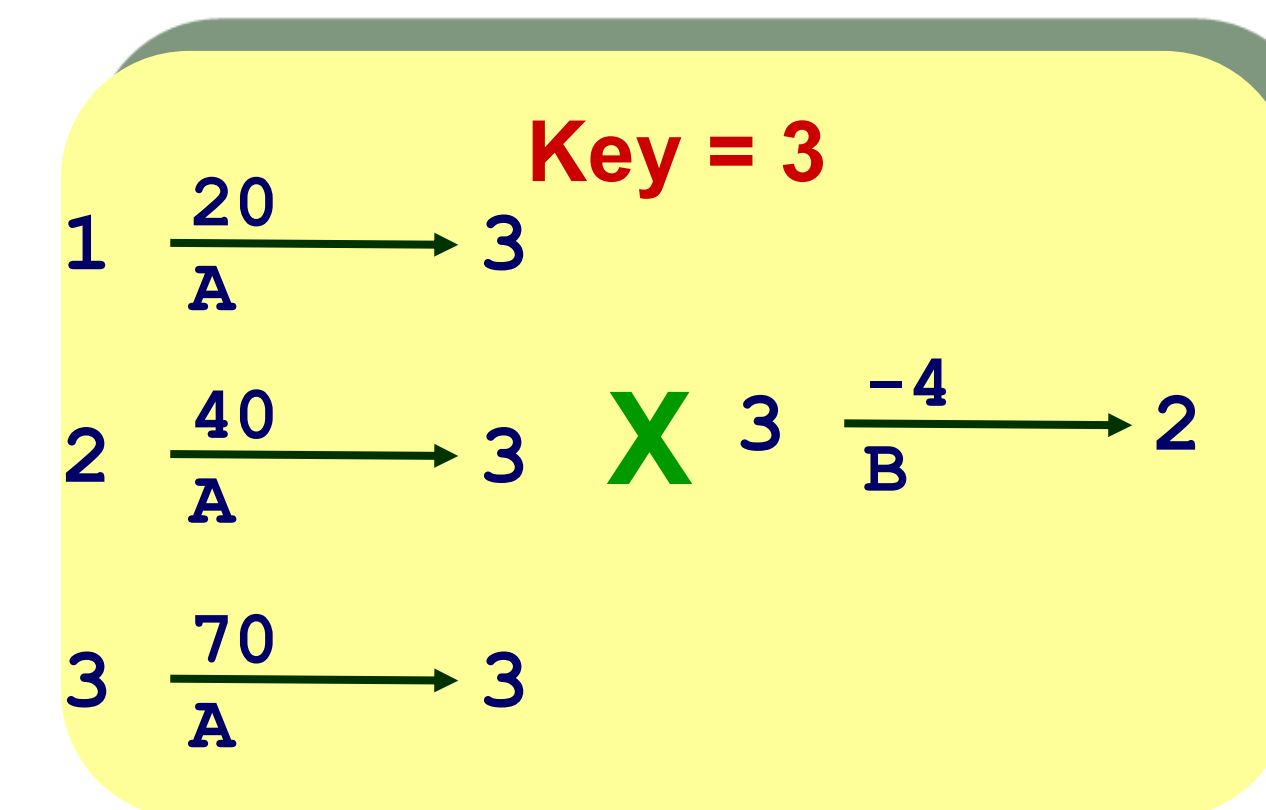
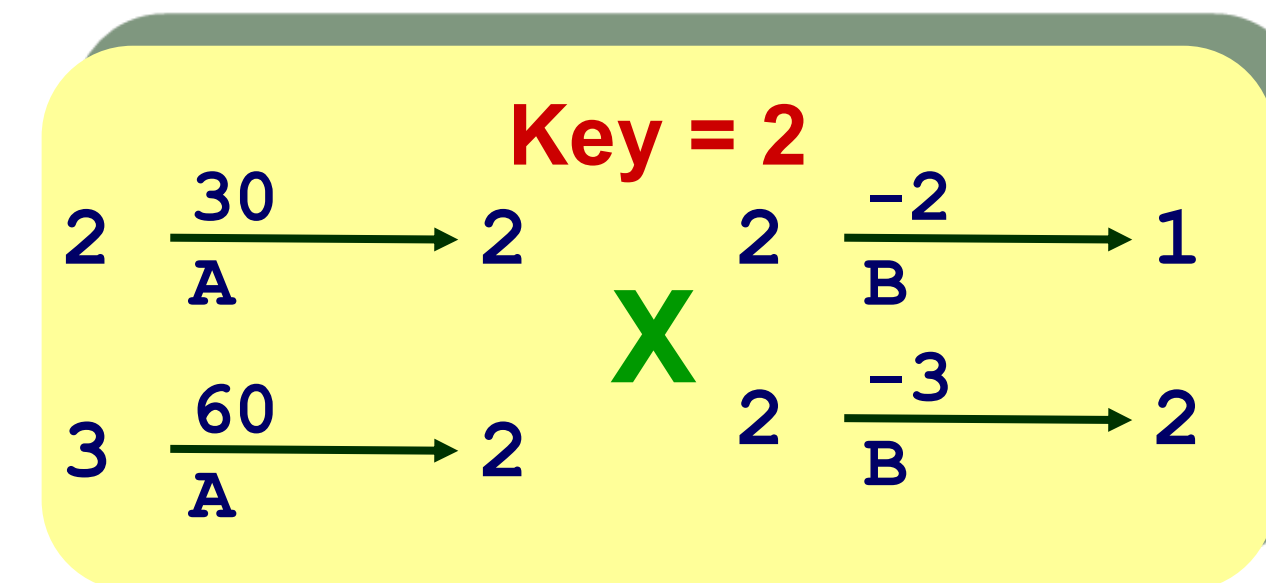
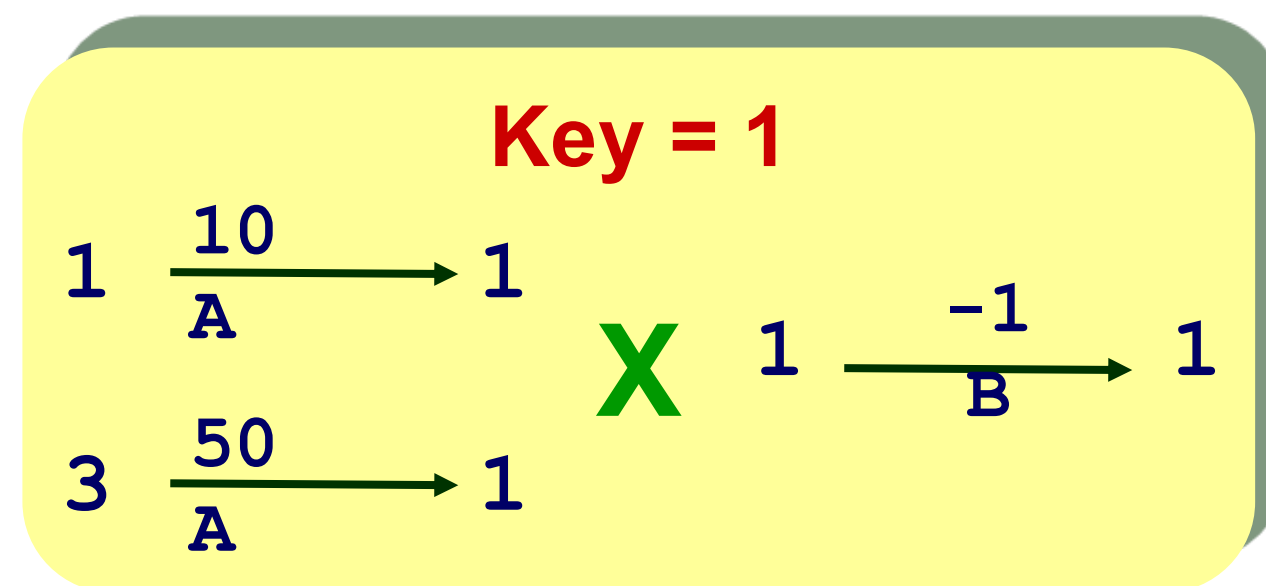
- Represent matrix as list of nonzero entries
<row, col, value, matrixID>
- How to represent the computation as map-reduce?
 - Phase 1: Compute all products $a_{i,k} \cdot b_{k,j}$
 - Phase 2: Sum products for each entry i,j
 - Each phase involves a Map/Reduce

Phase 1 Map of Matrix Multiply



- Group values $a_{i,k}$ and $b_{k,j}$ according to key k

Phase 1 “Reduce” of Matrix Multiply



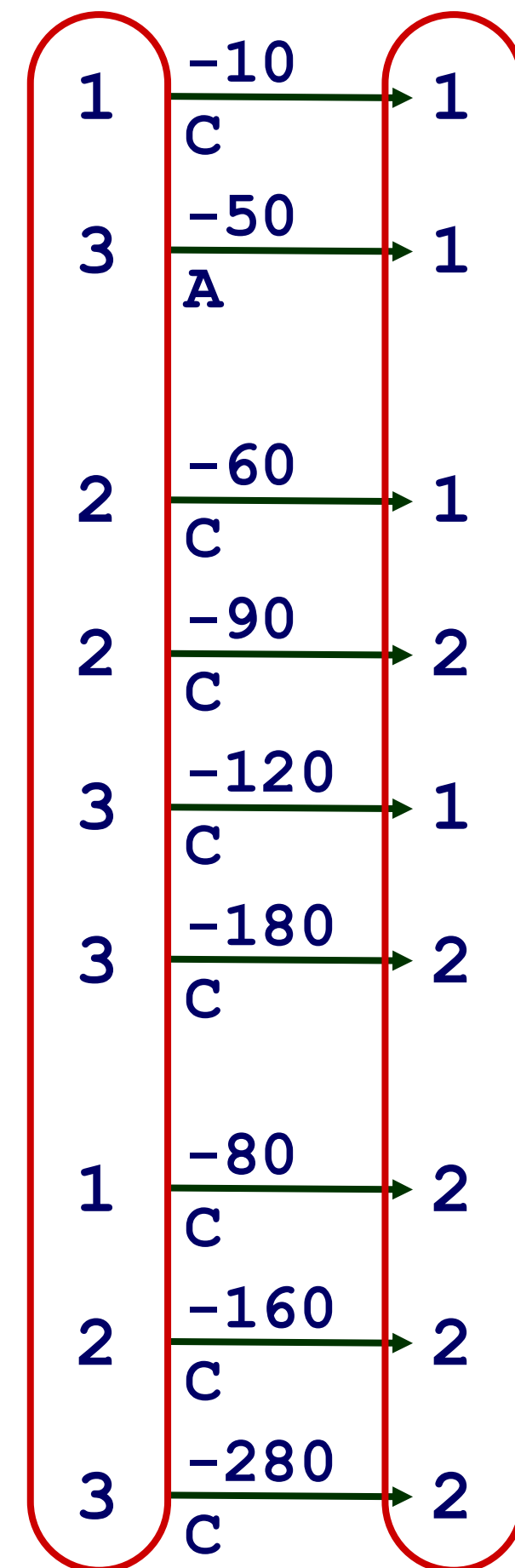
$$\begin{array}{lcl} 1 & \xrightarrow[C]{-10} & 1 \\ 3 & \xrightarrow[A]{-50} & 1 \end{array}$$

$$\begin{array}{lcl} 2 & \xrightarrow[C]{-60} & 1 \\ 2 & \xrightarrow[C]{-90} & 2 \\ 3 & \xrightarrow[C]{-120} & 1 \\ 3 & \xrightarrow[C]{-180} & 2 \end{array}$$

$$\begin{array}{lcl} 1 & \xrightarrow[C]{-80} & 2 \\ 2 & \xrightarrow[C]{-160} & 2 \\ 3 & \xrightarrow[C]{-280} & 2 \end{array}$$

- Generate all products $a_{i,k} \cdot b_{k,j}$

Phase 2 Map of Matrix Multiply



Key = row,col

Key = 1,1 $1 \xrightarrow{\frac{-10}{C}} 1$

Key = 1,2 $1 \xrightarrow{\frac{-80}{C}} 2$

Key = 2,1 $2 \xrightarrow{\frac{-60}{C}} 1$

Key = 2,2 $2 \xrightarrow{\frac{-90}{C}} 2$

$2 \xrightarrow{\frac{-160}{C}} 2$

Key = 3,1 $3 \xrightarrow{\frac{-120}{C}} 1$

$3 \xrightarrow{\frac{-50}{A}} 1$

Key = 3,2 $3 \xrightarrow{\frac{-280}{C}} 2$

$3 \xrightarrow{\frac{-180}{C}} 2$

- Group products $a_{i,k} \cdot b_{k,j}$ with matching values of i and j

Phase 2 Reduce of Matrix Multiply

Key = 1,1 $1 \xrightarrow[\text{C}]{-10} 1$

Key = 1,2 $1 \xrightarrow[\text{C}]{-80} 2$

Key = 2,1 $2 \xrightarrow[\text{C}]{-60} 1$

Key = 2,2 $2 \xrightarrow[\text{C}]{-90} 2$
 $2 \xrightarrow[\text{C}]{-160} 2$

Key = 3,1 $3 \xrightarrow[\text{C}]{-120} 1$
 $3 \xrightarrow[\text{A}]{-50} 1$

Key = 3,2 $3 \xrightarrow[\text{C}]{-280} 2$
 $3 \xrightarrow[\text{C}]{-180} 2$

$1 \xrightarrow[\text{C}]{-10} 1$

$1 \xrightarrow[\text{C}]{-80} 2$

$2 \xrightarrow[\text{C}]{-60} 1$

$2 \xrightarrow[\text{C}]{-250} 2$

$3 \xrightarrow[\text{C}]{-170} 1$

$3 \xrightarrow[\text{C}]{-460} 2$

C

$$\begin{bmatrix} -10 & -80 \\ -60 & -250 \\ -170 & -460 \end{bmatrix}$$

- Sum products to get final entries

Recap: MapReduce Implementation

Built on Top of Parallel File System

- Google: GFS, Hadoop: HDFS
- Provides global naming
- Reliability via replication (typically 3 copies)

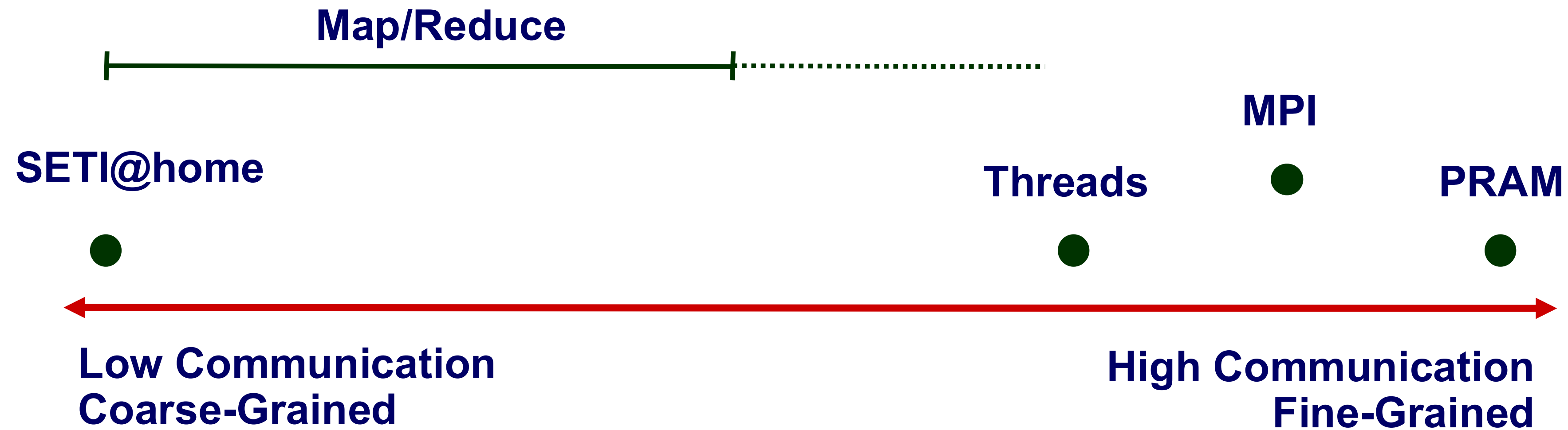
Breaks work into tasks

- Master schedules tasks on workers dynamically
- Typically $\#tasks \gg \#processors$

Net Effect

- Input: Set of files in reliable file system
- Output: Set of files in reliable file system

Exploring Parallel Computation Models



Map/Reduce Provides Coarse-Grained Parallelism

- Computation done by independent processes
- File-based communication

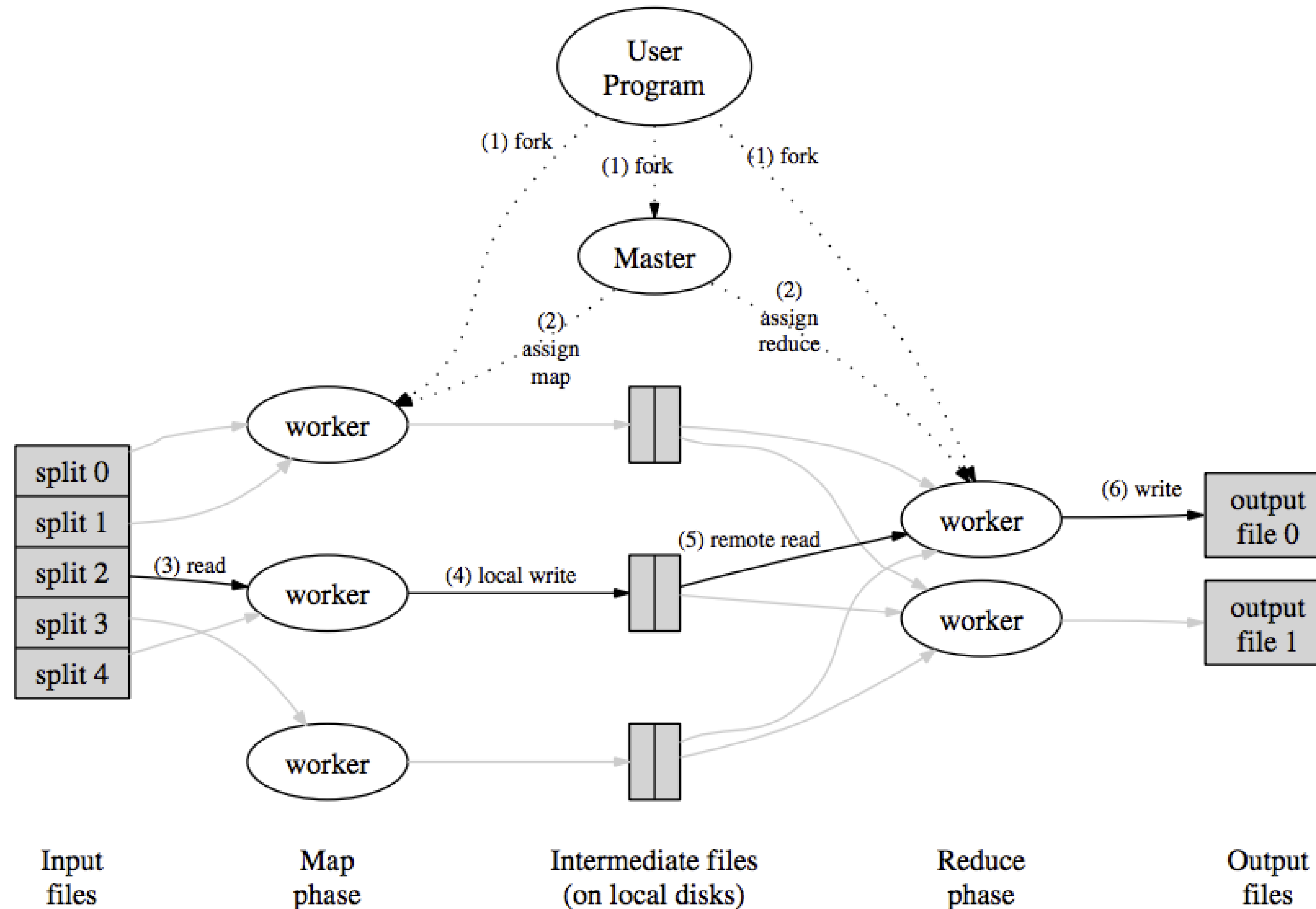
Observations

- Relatively “natural” programming model
- Research issue to explore full potential and limits

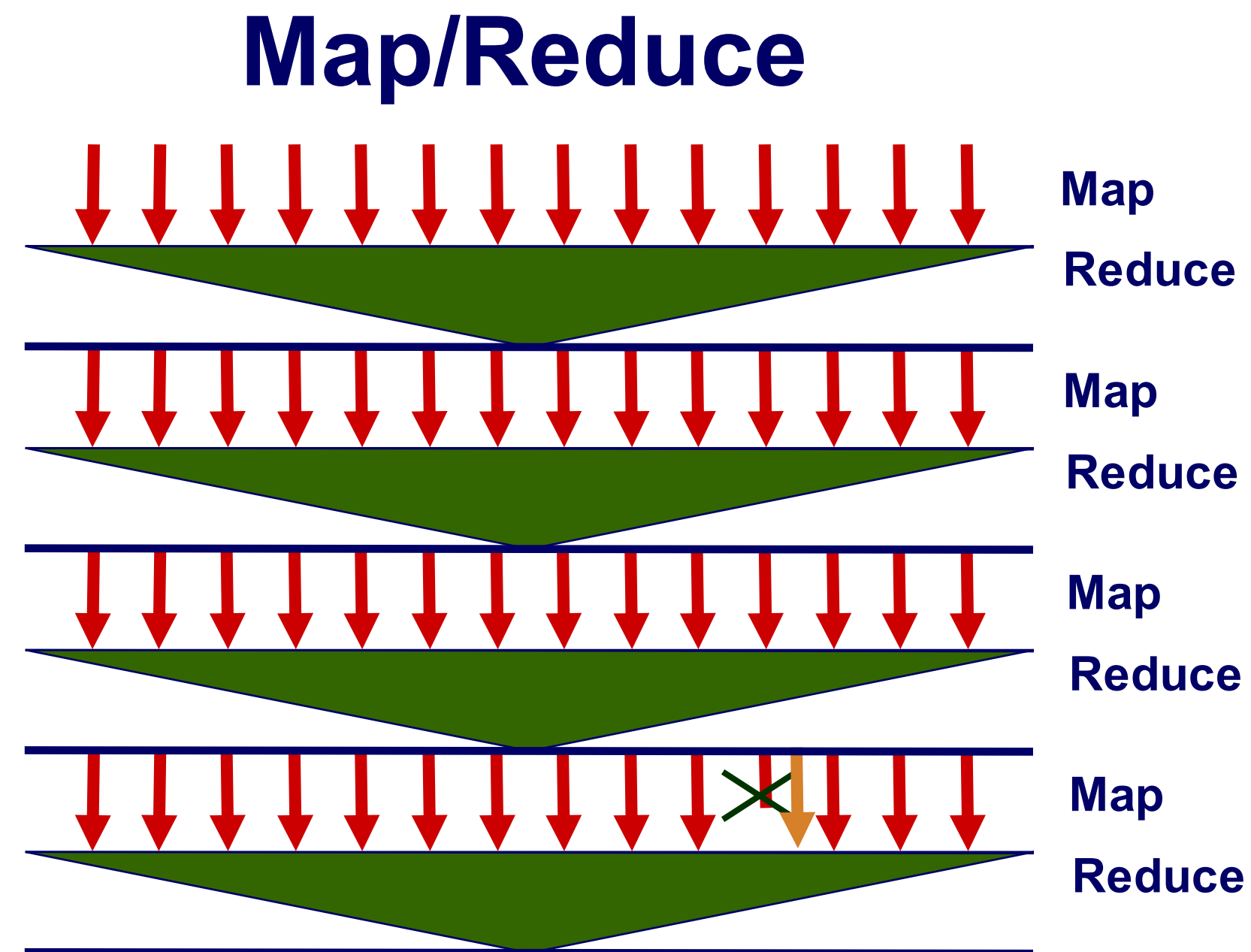
Today's topic: Batch Processing

- Overview
- IO & Unix pipes
- MapReduce
 - HDFS - infrastructure
 - Programming models (API)
 - Job execution (runtime)
 - Workflow
 - MapReduce Recap
- Beyond MapReduce

MapReduce System architecture (Paper)



Fault Tolerance



Data Integrity

- Store multiple copies of each file
- Including intermediate results of each Map / Reduce
 - Continuous checkpointing

Recovering from Failure

- Simply recompute lost result
 - Localized effect
- Dynamic scheduler keeps all processors busy

Map/Reduce Summary

Typical Map/Reduce Applications

- Sequence of steps, each requiring map & reduce
- Series of data transformations

Strengths of Map/Reduce

- User writes simple functions, system manages complexities of mapping, synchronization, fault tolerance
- Very general
- Good for large-scale data analysis

Map Reduce Summary: Cons

- Disk I/O overhead is super high
- Not flexible enough: Each map/reduce step must complete before next begins
- Not suitable for workloads:
 - Iterative processing
 - Real-time processing
- Map-reduce is still difficult to program with

PageRank Computation

Initially

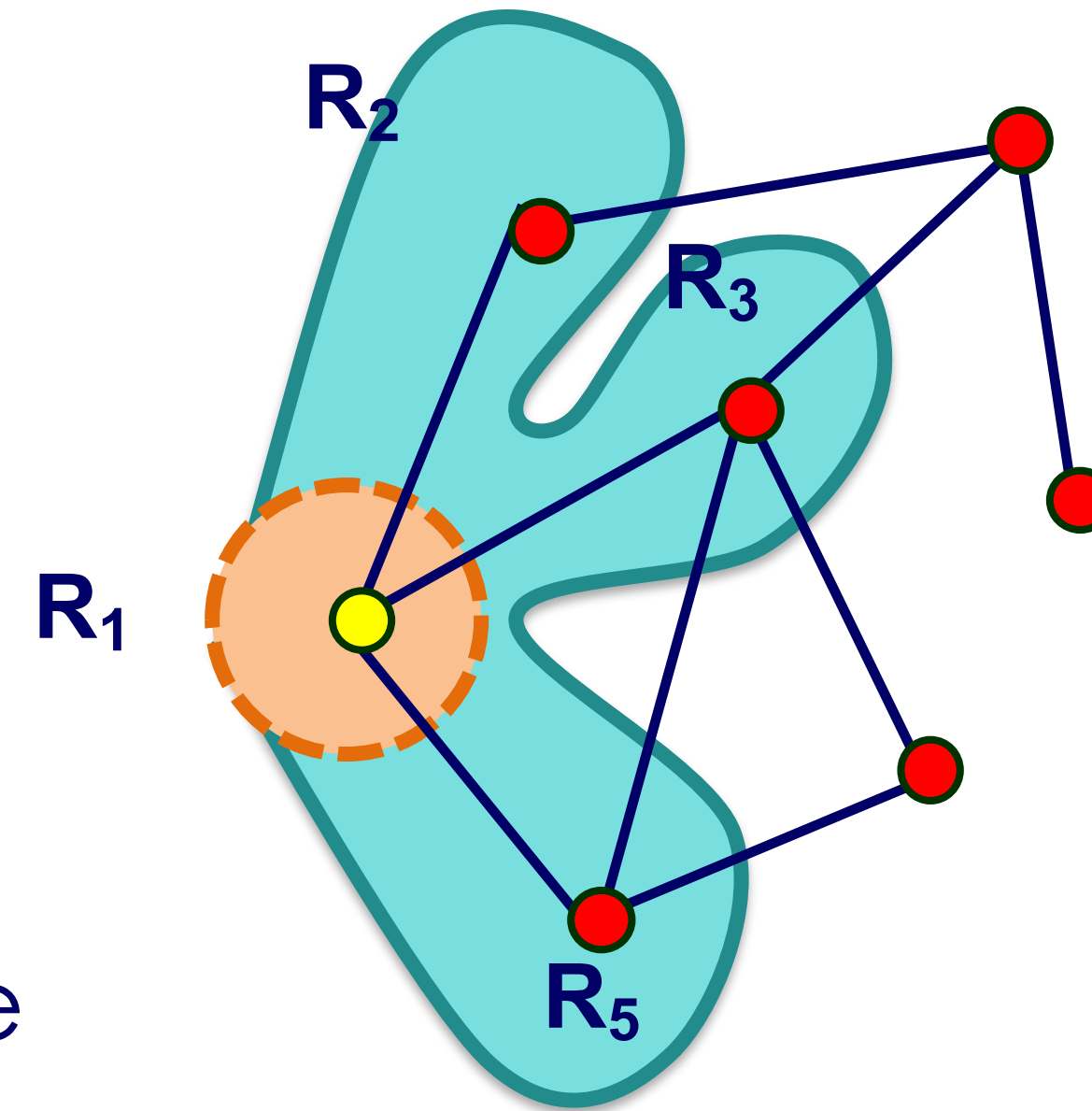
- Assign weight 1.0 to each page

Iteratively

- Select arbitrary node and update its value

Convergence

- Results unique, regardless of selection ordering



$$R_1 \leftarrow 0.1 + 0.9 * (\frac{1}{2} R_2 + \frac{1}{4} R_3 + \frac{1}{3} R_5)$$

Q: how to express pagerank using map-reduce?